Pre-Warm-Up Exercise 1: MNIST Classification Using ResNet-50

Jan Cuesta
Arizona State University

Abstract

This experiment replaces the simple convolutional neural network in the provided MNIST PyTorch tutorial with a standard ResNet-50 architecture. Because MNIST images are 28×28 *grayscale images, the ResNet-50 input and output layers were adapted while the residual architecture was preserved. The network was trained from scratch under three configurations to examine the influence of learning rate and training duration. Reducing the Adam learning rate from* 0.01 *to* 0.001 *improved the final test accuracy from 98.55% to 99.21% and reduced final test loss. Extending training from 10 to 15 epochs at a learning rate of* 0.001 *produced only a negligible additional improvement, reaching 99.22% final test accuracy.*

1. Introduction

Convolutional neural networks (CNNs) learn spatial features directly from image pixels by applying convolutional filters across successive layers. Early layers typically capture simple local patterns, while deeper layers combine these responses into more complex representations that can be used for classification.

The provided exercise follows a PyTorch CNN workflow on the MNIST handwritten-digit dataset [3]. The objective of this experiment was to replace the tutorial's simple CNN with a standard ResNet-50 architecture and evaluate its performance on the same ten-class classification problem.

ResNet introduced residual connections to make substantially deeper neural networks easier to optimize [1]. Instead of requiring a group of layers to learn a complete transformation directly, a residual block learns a residual function $F(x)$ and combines it with the original input:

$$y = F(x) + x. \tag{1}$$

The skip connection allows information and gradients to propagate through very deep networks more effectively.

In addition to implementing ResNet-50, three training configurations were compared to observe the influence of learning rate and number of epochs. Learning rate is an important optimization hyperparameter because it controls the magnitude of parameter updates during training [4].

2. Methods

2.1. Dataset and preprocessing

MNIST contains 60,000 training images and 10,000 test images of handwritten digits from 0 to 9 [3]. Each image is a 28×28 grayscale image. The images were converted to PyTorch tensors using `ToTensor()`, producing inputs with dimensions $1 \times 28 \times 28$.

Training and test data were loaded in batches of 100 images. The training data were shuffled before being divided into batches.

2.2. ResNet-50 adaptation

The official TorchVision implementation of ResNet-50 was used [2]. Pretrained weights were not loaded:

```
resnet50(weights=None)
```

Therefore, this experiment did not use transfer learning; all network parameters were learned from the MNIST training data.

The standard ResNet-50 architecture was originally designed for larger three-channel RGB images. Three modifications were therefore made for MNIST. First, the initial convolution was changed to accept one grayscale channel and to use a 3×3 kernel with stride 1 and padding 1. Second, the initial max-pooling layer was removed to avoid excessive spatial downsampling of the small 28×28 input. Finally, the last fully connected layer was replaced with a ten-output layer corresponding to the ten MNIST digit classes.

The internal residual structure of ResNet-50 was otherwise preserved. ResNet-50 uses bottleneck residual blocks composed of 1×1, 3×3, and 1×1 convolutions [1].

2.3. Training experiments

Cross-entropy loss was used as the classification objective and Adam was used for optimization. A new ResNet-50 model and a new optimizer were created for each experiment. The random seed was reset to 42 before each run so

Table 1. ResNet-50 performance for the three training configurations.

Configuration	Best acc.	Final acc.	Final loss
LR 0.01, 10 ep.	99.26% (8)	98.55%	0.0507
LR 0.001, 10 ep.	99.21% (10)	99.21%	0.0252
LR 0.001, 15 ep.	99.27% (7)	99.22%	0.0262



Figure 1. Test loss across epochs for the three ResNet-50 configurations.

that the experiments began from comparable initial random conditions.

Three configurations were evaluated:

- learning rate 0.01, trained for 10 epochs;
- learning rate 0.001, trained for 10 epochs;
- learning rate 0.001, trained for 15 epochs.

The first two experiments isolate the effect of reducing the learning rate while keeping the training duration constant. The third experiment keeps the lower learning rate and increases the number of complete passes through the training set.

3. Results

All three configurations achieved high classification accuracy, but their convergence behavior differed. Table 1 summarizes the final and best test performance.

Reducing the learning rate from 0.01 to 0.001 while maintaining 10 epochs increased final test accuracy by 0.66 percentage points, from 98.55% to 99.21%. The final test loss also decreased from 0.0507 to 0.0252.

The 15-epoch experiment produced a final accuracy of 99.22%, only 0.01 percentage points higher than the 10-epoch experiment with the same learning rate. Its highest test accuracy was 99.27%, reached at epoch 7.

4. Discussion

The results indicate that learning rate had a stronger effect on the final performance than the additional five training

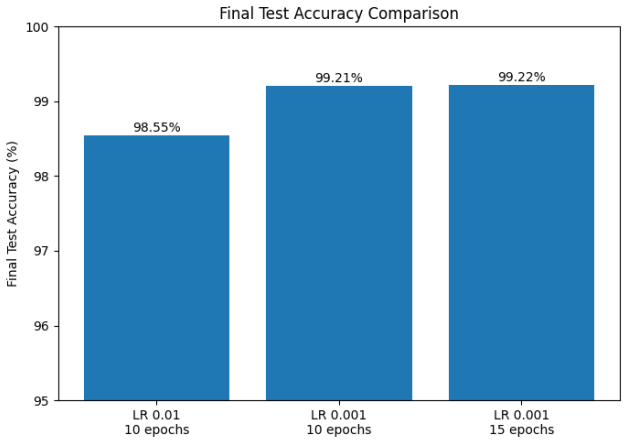


Figure 2. Final test accuracy for the three ResNet-50 configurations.

epochs. The baseline learning rate of 0.01 reached a high peak accuracy of 99.26% at epoch 8, but its test loss showed larger oscillations and its final accuracy decreased to 98.55%. In comparison, a learning rate of 0.001 finished the 10-epoch run with both higher accuracy and approximately half the test loss. This behavior is consistent with the importance of selecting an appropriate learning rate for stable optimization [4].

Increasing training duration from 10 to 15 epochs at the same learning rate did not provide a meaningful improvement in final classification accuracy. The final values were 99.21% and 99.22%, respectively. The 15-epoch run achieved its best accuracy earlier, at epoch 7, and its minimum test loss at epoch 10. Later epochs produced fluctuations rather than a sustained improvement. This suggests that the model had already reached near-saturated performance by approximately 10 epochs.

The longer experiment does not provide strong evidence of persistent overfitting. Overfitting would normally be indicated by continued improvement in training performance accompanied by a sustained deterioration in test performance. Although individual test-loss spikes occurred, test accuracy recovered and remained close to 99% through the final epoch. The results therefore support diminishing returns from additional training more clearly than they support a strong overfitting conclusion.

The experiment also illustrates why adapting the architecture to the input data is important. Applying the standard ImageNet input stem directly to a 28×28 grayscale image would both create a channel mismatch and reduce the spatial resolution very rapidly. Modifying the first convolution and removing the initial max-pooling layer allowed the residual network to operate directly on MNIST without resizing the images to ImageNet dimensions.

Finally, each configuration was evaluated with a single training run. The comparisons are therefore descriptive

rather than statistical; repeated runs would be required to estimate variability and determine whether very small differences such as 99.21% versus 99.22% are reproducible.

5. Conclusion

A ResNet-50 architecture was successfully adapted and trained from scratch for MNIST digit classification. The network was modified to accept $1 \times 28 \times 28$ grayscale inputs and to produce ten output classes. Among the tested configurations, reducing the Adam learning rate from 0.01 to 0.001 produced a more favorable final result, increasing final test accuracy from 98.55% to 99.21% and reducing test loss. Extending the lower-learning-rate experiment from 10 to 15 epochs produced almost no additional final accuracy improvement. For this task, a learning rate of 0.001 with approximately 10 epochs was sufficient to achieve about 99.2% test accuracy.

References

[1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, June 2016.

[2] PyTorch Contributors. Resnet – torchvision models. Torchvision Documentation, 2026. Accessed: 2026-08-23.

[3] Nutan Bhogendra Sharma. Pytorch convolutional neural network with mnist dataset. Medium, May 2021. Accessed: 2026-08-23.

[4] Leslie N. Smith. A disciplined approach to neural network hyper-parameters: Part 1 – learning rate, batch size, momentum, and weight decay. *arXiv preprint arXiv:1803.09820*, 2018.